Step 1 : Download the Arduino IDE form this link - Arduino IDE
Step 2 : Install the Arduino IDE in Your laptop
Step 3 : Set up the esp32 into the Arduino IDE
Step 4 : Click **File** option in the Arduino IDE after that click **Preferences** and after that click the **Additional boards manager URLs** and **Past** the bellow links
File → Preferences → Additional boards manager URLs → Paste(Below Links)
Link:
→ https://dl.espressif.com/dl/package_esp32_index.json
→ https://espressif.github.io/arduino-esp32/package_esp32_index.json
→https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
Step 5 : Here You taken the esp32 module it's working or not working check first using Arduino IDE
Step 6 : Click the **File** option after that click the **New Sketch** and after that Paste Below code
File → New Sketch → Paste(below code)
Code: Checking the esp32 working or not working

```
int ledPin = 2; // LED connected to GPIO 2

void setup() {
  pinMode(ledPin, OUTPUT); // Set GPIO 2 as an output
}

void loop() {
  digitalWrite(ledPin, HIGH); // Turn the LED on
  delay(1000);                // Wait for 1 second
  digitalWrite(ledPin, LOW);  // Turn the LED off
  delay(1000);                // Wait for 1 second
}
Output:
One second by second the inbuilt LED On and Off
```

Step 7 : Set the Select Other Board and Port. The board name is **ESP32 Dev Module** and **PORT** in case the PORT is not showing in YOUR computer or Laptop just download the **CP210x driver** and **CH340 driver**
CP210x Driver Link - Reference video
CH340 Driver Link - Reference video
Step 8 : First Verify the code to find the Errors and Rectify. Click Verify option
Step 9 : Without Errors Upload the Code to ESP32. Click upload option
Step 10 : In case the Code is uploaded output comes esp32 is perfectly working and the output does not come esp32 not working.
Step 11 : **Exercise - 1 : Blinking the led 4 seconds**
Code: →

```
int ledPin = 2; // LED connected to GPIO 2

void setup() {
  pinMode(ledPin, OUTPUT); // Set GPIO 2 as an output
}

void loop() {
  digitalWrite(ledPin, HIGH); // Turn the LED on
  delay(4000);                // Wait for 1 second
  digitalWrite(ledPin, LOW);  // Turn the LED off
  delay(4000);                // Wait for 1 second
}
Output:
4 seconds by 4 seconds the inbuilt LED On and Off
```

Step 12 : Explain the code

This code makes an LED blink repeatedly. The LED is connected to pin 2 of the microcontroller. In the `setup()` function, the pin is set as an output so it can control the LED. In the `loop()` function, the LED is turned on for 4 seconds and then turned off for 4 seconds. This on-and-off blinking cycle continues indefinitely.

Step 13 : Click **File** option in the Arduino IDE after that click **Preferences** and after that click the **Additional boards manager URLs** and **Past** the bellow links

File → Preferences → Additional boards manager URLs → Paste(Below Links)

→https://raw.githubusercontent.com/ricardoquesada/esp32-arduino-lib-builder/master/bluepad32_files/package_esp32_bluepad32_index.json

Step 14 : Set the Select Other Board and Port. The board name is **ESP32 Dev Module**(Bluepad) **OR ESP32 Wrover Module**(Bluepad) and **PORT**

Step 15 : Exercise - 2 : ESP32 Blinking led | Serial Monitor output using L1, L2, R1, R2 for ps4 controller

Code :

```
#include <Bluepad32.h>

// Define LED pin
int ledPin = 2;  // LED pin
GamepadPtr myGamepad;
bool ledState = LOW;  // Variable to store LED state

void setup() {
  // Set LED pin as output
```

```
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, ledState);  // Initialize LED to be off

  Serial.begin(115200);
  // Initialize Bluepad32 library
  BP32.setup(&onConnect, &onDisconnect);
  Serial.println("Waiting for PS4 controller to connect...");
}

void loop() {
  BP32.update();  // Update the controller state
  if (myGamepad) {
    // Check if L1 button is pressed
    if (myGamepad->l1()) {
      Serial.println("L1 button pressed");
      // Toggle LED state
      ledState = !ledState;
      digitalWrite(ledPin, ledState);  // Update LED state
       // Wait for button release to avoid multiple toggles from a single
press
      while (myGamepad->l1()) {
        BP32.update();  // Keep updating controller state
      }
    }

    // Check if R1 button is pressed
    if (myGamepad->r1()) {
      Serial.println("R1 button pressed");
      // Set LED state to LOW
      ledState = LOW;  // Set LED state to LOW
      digitalWrite(ledPin, ledState);  // Update LED state
       // Wait for button release to avoid multiple toggles from a single
press
      while (myGamepad->r1()) {
        BP32.update();  // Keep updating controller state
      }
    }

    if (myGamepad->l2()) {
      Serial.println("L2 button pressed");
```

```
      // Toggle LED state
      ledState = !ledState;
      digitalWrite(ledPin, ledState);  // Update LED state
       // Wait for button release to avoid multiple toggles from a single
press
      while (myGamepad->l1()) {
        BP32.update();  // Keep updating controller state
      }
    }

    // Check if R1 button is pressed
    if (myGamepad->r2()) {
      Serial.println("R2 button pressed");
      // Set LED state to LOW
      ledState = LOW;  // Set LED state to LOW
      digitalWrite(ledPin, ledState);  // Update LED state
       // Wait for button release to avoid multiple toggles from a single
press
      while (myGamepad->r1()) {
        BP32.update();  // Keep updating controller state
      }
    }
  }
  delay(20);  // Small delay to make the code run smoother
}

// Called when a Gamepad connects
void onConnect(GamepadPtr gp) {
  Serial.println("Controller connected!");
  myGamepad = gp;
}

// Called when a Gamepad disconnects
void onDisconnect(GamepadPtr gp) {
  Serial.println("Controller disconnected!");
  myGamepad = nullptr;
}
```

Output :

L1 button pressed: The LED toggles (turns on if it was off, or off if it was on).

R1 button pressed: The LED turns off.

L2 button pressed: The LED toggles (same as L1).
R2 button pressed: The LED turns off.

Step 16 : Explanation

This code uses the Bluepad32 library to control an LED with a PS4 controller. The LED is connected to pin 2 of the microcontroller. When the program starts, it initializes the Bluepad32 library and waits for a controller to connect. Once connected, it checks for specific button presses on the controller. Pressing the **L1** button toggles the LED's state (on or off), while pressing the **R1** button turns the LED off. Similarly, the **L2** button toggles the LED state, and the **R2** button turns it off. The program ensures the LED state changes only once per button press by waiting for the button to be released before checking again. The LED state updates in real-time, and the loop continuously monitors the controller's input.

Step 17 : Exercise 3 - ESP32 Blinking led | Serial Monitor output using Joystick Left and Right:

Code :

```
#include <Bluepad32.h>


// Define LED pin

int ledPin = 2;  // LED pin

GamepadPtr myGamepad;


// Timer variables

unsigned long ledOnTime = 0; // Time when the LED was turned on

const unsigned long ledDuration = 2000; // Duration to keep the LED on (in
milliseconds)


void setup() {

  // Set LED pin as output

  pinMode(ledPin, OUTPUT);
```

```
  Serial.begin(115200);

  // Initialize Bluepad32 library
  BP32.setup(&onConnect, &onDisconnect);

  Serial.println("Waiting for PS4 controller to connect...");
}

void loop() {
  BP32.update();  // Update the controller state

  if (myGamepad) {
    // Read the joystick positions for control
    int16_t rightStickY = myGamepad->axisRY(); // Right joystick vertical movement
    int16_t rightStickX = myGamepad->axisRX(); // Right joystick horizontal movement
    int16_t leftStickY = myGamepad->axisY();   // Left joystick vertical movement
    int16_t leftStickX = myGamepad->axisX();   // Left joystick horizontal movement

    // Control the LED based on right joystick vertical movement
    if (abs(rightStickY) > 20) { // Vertical movement detected
```

```
    digitalWrite(ledPin, HIGH);  // Turn the LED on
    ledOnTime = millis(); // Record the time the LED was turned on
    Serial.println("Right joystick moved vertically."); // Serial print
  }

  // Control the LED based on left joystick vertical movement
  if (abs(leftStickY) > 20) { // Vertical movement detected
    digitalWrite(ledPin, HIGH);  // Turn the LED on
    ledOnTime = millis(); // Record the time the LED was turned on
    Serial.println("Left joystick moved vertically."); // Serial print
  }

  // Control the LED based on horizontal movements of either joystick
  if (abs(rightStickX) > 20) { // Horizontal movement detected
    digitalWrite(ledPin, LOW);   // Turn the LED off
      Serial.println("Right joystick moved horizontally."); // Serial
print
  }
  if (abs(leftStickX) > 20) { // Horizontal movement detected
    digitalWrite(ledPin, LOW);   // Turn the LED off
    Serial.println("Left joystick moved horizontally."); // Serial print
  }
```

```
    // Turn off the LED after the specified duration, unless horizontal movement is detected
        if (digitalRead(ledPin) == HIGH && (millis() - ledOnTime >= ledDuration)) {
      digitalWrite(ledPin, LOW); // Turn the LED off after duration
    }
  }

  delay(20);  // Small delay to make the code run smoother
}

// Called when a Gamepad connects
void onConnect(GamepadPtr gp) {
  Serial.println("Controller connected!");
  myGamepad = gp;
}

// Called when a Gamepad disconnects
void onDisconnect(GamepadPtr gp) {
  Serial.println("Controller disconnected!");
  myGamepad = nullptr;
}
```

Output:
Right joystick moved up/down: The LED turns on for 2 seconds. Serial monitor shows: "Right joystick moved vertically."

Left joystick moved up/down: The LED turns on for 2 seconds. Serial monitor shows: "Left joystick moved vertically."
Right joystick moved left/right: The LED turns off immediately. Serial monitor shows: "Right joystick moved horizontally."
Left joystick moved left/right: The LED turns off immediately. Serial monitor shows: "Left joystick moved horizontally."
Step 18 : Explanation:
This code controls an LED using a PS4 controller's joysticks, connected via the Bluepad32 library. The LED is connected to pin 2. When the right or left joystick is moved vertically (up or down), the LED turns on for 2 seconds, and the action is logged in the serial monitor. If either joystick is moved horizontally (left or right), the LED turns off immediately, and the corresponding message is displayed in the serial monitor. The program continuously checks the joystick positions, updates the LED state based on movement, and ensures the LED turns off automatically after 2 seconds unless a horizontal movement is detected. This creates a simple interaction between joystick movements and the LED.
Step 19 : ESP32 Blinking led | Serial Monitor output using Arrows:
Code:

```
#include <Bluepad32.h>

const int ledPin = 2; // LED connected to GPIO 2
GamepadPtr myGamepad;

Bluepad32 bp32; // Declare Bluepad32 as an object

void onConnect(GamepadPtr gp) {
  myGamepad = gp;
  Serial.println("Controller connected!");
}

void onDisconnect(GamepadPtr gp) {
  myGamepad = nullptr;
  Serial.println("Controller disconnected!");
}

void setup() {
  Serial.begin(115200);
  pinMode(ledPin, OUTPUT);

  bp32.setup(&onConnect, &onDisconnect); // Use bp32 object to call setup
}

void loop() {
```

```
  bp32.update(); // Use bp32 object to call update

  // Check if controller is connected
  if (myGamepad && myGamepad->isConnected()) {
    bool ledState = LOW; // Default LED state

    // Check D-pad state
    int dpadState = myGamepad->dpad(); // Get the current D-pad state

    if (dpadState & DPAD_UP) {
      Serial.println("Up button pressed");
      ledState = HIGH; // Turn LED on
    }
    if (dpadState & DPAD_DOWN) {
      Serial.println("Down button pressed");
      ledState = HIGH; // Turn LED on
    }
    if (dpadState & DPAD_LEFT) {
      Serial.println("Left button pressed");
      ledState = HIGH; // Turn LED on
    }
    if (dpadState & DPAD_RIGHT) {
      Serial.println("Right button pressed");
      ledState = HIGH; // Turn LED on
    }

    digitalWrite(ledPin, ledState); // Set LED to the determined state
  } else {
      digitalWrite(ledPin, LOW); // Turn LED off if controller is not
connected
  }
}
```

Output :

The LED turns **on** when any D-pad button (Up, Down, Left, Right) is pressed, and the serial monitor shows which button was pressed. The LED turns **off** when no button is pressed or the controller is disconnected. The serial monitor also displays messages when the controller connects or disconnects.

Step 20 : Explanation

This code controls an LED using the D-pad buttons on a connected game controller. When the controller is connected, the code checks for button presses (Up, Down, Left, Right) on the D-pad. If any button is pressed, the LED turns on, and a message indicating which button was

pressed is shown in the serial monitor. If no button is pressed or the controller is disconnected, the LED turns off. The serial monitor also displays messages when the controller connects or disconnects. The Bluepad32 library is used to manage the controller connection and update the button states.

Step 21 : ESP32 Blinking led | Serial monitor output using Box, Triangle, Circle, X:

Code :

```
#include <Bluepad32.h>

const int ledPin = 2; // LED connected to GPIO 2
GamepadPtr myGamepad;

Bluepad32 bp32; // Declare Bluepad32 as an object

void onConnect(GamepadPtr gp) {
  myGamepad = gp;
  Serial.println("Controller connected!");
}

void onDisconnect(GamepadPtr gp) {
  myGamepad = nullptr;
  Serial.println("Controller disconnected!");
}
void setup() {
  Serial.begin(115200);
  pinMode(ledPin, OUTPUT);

  bp32.setup(&onConnect, &onDisconnect); // Use bp32 object to call setup
}

void loop() {
  bp32.update(); // Use bp32 object to call update

  // Check if controller is connected
if (myGamepad && myGamepad->isConnected()) {
    bool ledState = LOW; // Default LED state

    // Check each button individually and print its name
    if (myGamepad->a()) {
      Serial.println("X button pressed");
      ledState = HIGH; // Turn LED on
```

```
    }
    if (myGamepad->b()) {
      Serial.println("Circle button pressed");
      ledState = HIGH; // Turn LED on
    }
    if (myGamepad->x()) {
      Serial.println("Square button pressed");
      ledState = HIGH; // Turn LED on
    }
    if (myGamepad->y()) {
      Serial.println("Triangle button pressed");
      ledState = HIGH; // Turn LED on
    }

    digitalWrite(ledPin, ledState); // Set LED to the determined state
  } else {
      digitalWrite(ledPin, LOW); // Turn LED off if controller is not
connected
  }
}
```

Output :

The LED turns **on** when any of the buttons (X, Circle, Square, Triangle) on the controller are pressed. The serial monitor shows which button was pressed. The LED turns **off** when no buttons are pressed or if the controller is disconnected. The serial monitor also shows when the controller connects or disconnects.

Step 22 : Explanation

This code controls an LED using the buttons (X, Circle, Square, Triangle) on a connected game controller. When the controller is connected, it checks if any of these buttons are pressed. If a button is pressed, the LED turns **on** and the serial monitor shows which button was pressed. If no button is pressed, or the controller is disconnected, the LED turns **off**. The serial monitor also displays messages when the controller connects or disconnects. The Bluepad32 library is used to manage the controller connection and check the button states.

Step 23 : ESP32 Blinking led | Serial Motor output using Joystick Left and Right:

Code :

```
#include <Bluepad32.h>

// Define motor pins (using GPIO 12, 14, 27, and 26)

int motor1Pin1 = 12;  // Motor 1 direction pin 1
```

```
int motor1Pin2 = 14;  // Motor 1 direction pin 2
int motor2Pin1 = 27;  // Motor 2 direction pin 1
int motor2Pin2 = 26;  // Motor 2 direction pin 2
// Define the LED pin
int ledPin = 2;  // LED pin
GamepadPtr myGamepad;
void setup() {
  // Set motor pins as outputs
  pinMode(motor1Pin1, OUTPUT);
  pinMode(motor1Pin2, OUTPUT);
  pinMode(motor2Pin1, OUTPUT);
  pinMode(motor2Pin2, OUTPUT);
  // Set LED pin as output
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW);  // Initialize LED to OFF
  Serial.begin(115200);
  // Initialize Bluepad32 library
  BP32.setup(&onConnect, &onDisconnect);
  Serial.println("Waiting for PS4 controller to connect...");
}
void loop() {
  BP32.update();  // Update the controller state
  if (myGamepad) {
```

```
    // Left joystick controls motor based on vertical movement
     int leftJoystickY = myGamepad->axisY();   // Read vertical movement (-512 to +512)
     int leftJoystickX = myGamepad->axisX();  // Read horizontal movement (-512 to +512)
     if (abs(leftJoystickY) > 200) {   // Motor ON if vertical movement exceeds a threshold
      if (leftJoystickY > 0) {
        setMotorDirection(true);  // Forward direction
      } else {
        setMotorDirection(false);  // Reverse direction
      }
     } else if (abs(leftJoystickX) > 200) {   // Motor OFF if horizontal movement exceeds a threshold
      stopAllMotors();
    }


    // Right joystick controls LED
     int rightJoystickY = myGamepad->axisRY();  // Read vertical movement of right joystick
      if (abs(rightJoystickY) > 200) {   // LED ON if vertical movement exceeds a threshold
      digitalWrite(ledPin, HIGH);  // Turn ON LED
    } else {
      digitalWrite(ledPin, LOW);  // Turn OFF LED
    }
```

```
  }
  delay(20);  // Small delay to make the code run smoother
}
// Set direction for all motors
void setMotorDirection(bool forward) {
  if (forward) {
    // Set all motors to move in one direction
    digitalWrite(motor1Pin1, HIGH);
    digitalWrite(motor1Pin2, LOW);
    digitalWrite(motor2Pin1, HIGH);
    digitalWrite(motor2Pin2, LOW);
  } else {
    // Set all motors to move in the opposite direction
    digitalWrite(motor1Pin1, LOW);
    digitalWrite(motor1Pin2, HIGH);
    digitalWrite(motor2Pin1, LOW);
    digitalWrite(motor2Pin2, HIGH);
  }
}
// Stop all motors
void stopAllMotors() {
  digitalWrite(motor1Pin1, LOW);
  digitalWrite(motor1Pin2, LOW);
```

```
  digitalWrite(motor2Pin1, LOW);
  digitalWrite(motor2Pin2, LOW);
}
// Called when a Gamepad connects
void onConnect(GamepadPtr gp) {
  Serial.println("Controller connected!");
  myGamepad = gp;
}
// Called when a Gamepad disconnects
void onDisconnect(GamepadPtr gp) {
  Serial.println("Controller disconnected!");
  myGamepad = nullptr;
}
```

Output :

1. **Controller Connected:** Displays "Controller connected!"
2. **Controller Disconnected:** Displays "Controller disconnected!"

- **Left Joystick:**
  - Move up/down: Motors move forward/backward.
  - Move left/right: Motors stop.
- **Right Joystick:**
  - Move up/down: LED turns ON.
  - Release: LED turns OFF.

Step 24 : Explanation

This code allows you to control two motors and an LED using a PS4 controller connected to an ESP32. In the setup, the code initializes the motor pins and the LED pin. It waits for the PS4 controller to connect. Once connected, the left joystick is used to control the motors: moving it up or down makes the motors move forward or backward, while moving it left or right stops the motors. The right joystick is used to control the LED: moving it up turns the LED on, and moving it back to

the middle turns the LED off. The code continuously checks the joystick movements and updates the motor and LED states accordingly.

Step 25 : The Board, 1 BO Motor ps4 controller:

Code :

```
#include <Bluepad32.h>


// Define motor pins (using GPIO 12, 14, 27, and 26)
int motor1Pin1 = 12;  // Motor 1 direction pin 1
int motor1Pin2 = 14;  // Motor 1 direction pin 2
int motor2Pin1 = 27;  // Motor 2 direction pin 1
int motor2Pin2 = 26;  // Motor 2 direction pin 2


GamepadPtr myGamepad;


void setup() {
  // Set motor pins as outputs
  pinMode(motor1Pin1, OUTPUT);
  pinMode(motor1Pin2, OUTPUT);
  pinMode(motor2Pin1, OUTPUT);
  pinMode(motor2Pin2, OUTPUT);


  Serial.begin(115200);
```

```
  // Initialize Bluepad32 library
  BP32.setup(&onConnect, &onDisconnect);

  Serial.println("Waiting for PS4 controller to connect...");
}

void loop() {
  BP32.update();  // Update the controller state

  if (myGamepad) {
    // Read left joystick vertical position
    int16_t leftStickY = myGamepad->axisY(); // Vertical movement (up/down)

    // Motor control based on left joystick Y-axis
    if (abs(leftStickY) > 20) { // Deadband to avoid small joystick movements
      if (leftStickY > 0) {
        // Move motors forward
        setMotorDirection(true);
      } else {
        // Move motors backward
        setMotorDirection(false);
      }
```

```
    } else {
      // Stop all motors
      stopAllMotors();
    }
  }

  delay(20);  // Small delay to make the code run smoother
}

// Set direction for all motors
void setMotorDirection(bool forward) {
  if (forward) {
    // Set all motors to move in one direction
    digitalWrite(motor1Pin1, HIGH);
    digitalWrite(motor1Pin2, LOW);
    digitalWrite(motor2Pin1, HIGH);
    digitalWrite(motor2Pin2, LOW);
  } else {
    // Set all motors to move in the opposite direction
    digitalWrite(motor1Pin1, LOW);
    digitalWrite(motor1Pin2, HIGH);
    digitalWrite(motor2Pin1, LOW);
    digitalWrite(motor2Pin2, HIGH);
```

```
  }
}

// Stop all motors
void stopAllMotors() {
  digitalWrite(motor1Pin1, LOW);
  digitalWrite(motor1Pin2, LOW);
  digitalWrite(motor2Pin1, LOW);
  digitalWrite(motor2Pin2, LOW);
}

// Called when a Gamepad connects
void onConnect(GamepadPtr gp) {
  Serial.println("Controller connected!");
  myGamepad = gp;
}

// Called when a Gamepad disconnects
void onDisconnect(GamepadPtr gp) {
  Serial.println("Controller disconnected!");
  myGamepad = nullptr;
}
```

Output :

Motor Control: When you push the joystick up, the motors move forward. When you push it down, they move backward. If you don't move the joystick, the motors stop.

Controller Connection: The code waits for the PS4 controller to connect and prints a message when it's connected or disconnected.

Step 26 : Explanation

This code allows you to control two motors using a PS4 controller with the ESP32. When the program runs, it initializes the motor pins and sets up the PS4 controller using the Bluepad32 library. The controller's left joystick is used to control the movement of the motors. If the joystick is moved up, the motors move forward, and if moved down, the motors move backward. If the joystick is not moved, the motors stop. The code also manages the connection and disconnection of the PS4 controller, displaying messages when the controller is connected or disconnected. Essentially, the code lets you control the direction of the motors based on the vertical movement of the left joystick on the PS4 controller.

Step 27 : The Board, 2 BO Motors and 2 Servo Motors:

Code :

```
#include <Bluepad32.h>

#include <ESP32Servo.h>  // Servo library for ESP32


// Define the servo motor pins

int servoPin1 = 19;  // First servo motor pin

int servoPin2 = 21;  // Second servo motor pin (change pin number as
needed)

Servo myServo1;      // First servo object

Servo myServo2;      // Second servo object


GamepadPtr myGamepad;
```

```
void setup() {
  // Attach servos to the specified pins
  myServo1.attach(servoPin1);
  myServo2.attach(servoPin2);

  // Initialize both servos to 90 degrees
  myServo1.write(90);
  myServo2.write(90);

  Serial.begin(115200);

  // Initialize Bluepad32 library
  BP32.setup(&onConnect, &onDisconnect);

  Serial.println("Waiting for PS4 controller to connect...");
}

void loop() {
  BP32.update();  // Update the controller state

  if (myGamepad) {
    // Read left joystick position
    int16_t leftStickX = myGamepad->axisX(); // Horizontal movement
(left/right)
```

```
    int16_t leftStickY = myGamepad->axisY(); // Vertical movement
(up/down)


    // Control first servo motor based on left joystick X-axis
    if (abs(leftStickX) > 20) { // Deadband to avoid small joystick
movements
      if (leftStickX > 0) {
        myServo1.write(0);  // Rotate first servo to 0 degrees (right)
      } else {
        myServo1.write(180);  // Rotate first servo to 180 degrees (left)
      }
    } else {
      myServo1.write(90);  // Rotate first servo to 90 degrees (center)
    }


    // Control second servo motor based on left joystick Y-axis
    if (abs(leftStickY) > 20) { // Deadband to avoid small joystick
movements
      if (leftStickY > 0) {
        myServo2.write(0);  // Rotate second servo to 0 degrees (up)
      } else {
        myServo2.write(180);  // Rotate second servo to 180 degrees (down)
      }
    } else {
```

```
      myServo2.write(90);  // Rotate second servo to 90 degrees (center)
    }
  }

  delay(20);  // Small delay to make the code run smoother
}

// Called when a Gamepad connects
void onConnect(GamepadPtr gp) {
  Serial.println("Controller connected!");
  myGamepad = gp;
}

// Called when a Gamepad disconnects
void onDisconnect(GamepadPtr gp) {
  Serial.println("Controller disconnected!");
  myGamepad = nullptr;
}
```

Output :

This code allows two servo motors to be controlled using a PS4 controller connected to an ESP32. The servos are connected to GPIO pins 19 and 21 of the ESP32. When the left joystick is moved, the first servo motor rotates horizontally (left or right), while the second servo motor rotates vertically (up or down). The servos are set to their default positions (90 degrees) when the joystick is centered. The code updates the servo positions based on the joystick's movement, with a deadzone to avoid small movements. The PS4 controller must be connected

to the ESP32 for the control to work, and the Bluepad32 library handles the connection between the controller and the ESP32.

Step 28: Explanation

This code allows you to control two servos using a PS4 controller with the ESP32. It begins by including the necessary libraries for the controller (`Bluepad32`) and for servo control (`ESP32Servo`). The servo motors are connected to specific pins (19 and 21) on the ESP32, and they are initialized at a neutral position of 90 degrees. The `setup()` function configures these pins, attaches the servos to them, and establishes communication with the PS4 controller. The `loop()` function continuously checks the state of the controller, particularly the movements of the left joystick. When the left joystick is moved along the X-axis, it controls the first servo (rotating it left or right), and when the Y-axis is moved, it controls the second servo (moving it up or down). The servos return to a neutral position of 90 degrees if the joystick is not moved enough to exceed a threshold. The code also includes `onConnect()` and `onDisconnect()` functions to handle the connection and disconnection of the controller, ensuring the system responds accordingly.

Step 29 : The Board, 4 BO Motors and 5 Servo Motors

Code :

```
#include <Bluepad32.h>

#include <ESP32Servo.h>  // Use ESP32 Servo library


// Define the enable pins for the motors

const int motor1_enable_pin = 13;  // Motor 1 enable pin

const int motor2_enable_pin = 25;  // Motor 2 enable pin

// Define the motor control pins

const int motor1_forward_pin = 12;   // Motor 1 forward pin

const int motor1_backward_pin = 14;  // Motor 1 backward pin

const int motor2_forward_pin = 27;   // Motor 2 forward pin
```

```
const int motor2_backward_pin = 26;  // Motor 2 backward pin
const int buzzer = 15;               // Buzzer pin

// Define the motor control range
const int motor_range_max = 255;   // Maximum speed (forward)
const int motor_range_min = -255;  // Minimum speed (backward)

// Define the servo pins
const int servo1_pin = 2;   // Servo 1 pin (left joystick up and down)
const int servo2_pin = 4;   // Servo 2 pin (left joystick left and right)
const int servo3_pin = 5;   // Servo 3 pin (right joystick up and down)
const int servo4_pin = 18;  // Servo 4 pin (right joystick left and right)
const int servo5_pin = 19;  // Servo 5 pin (throttle and brake buttons)

// Initialize the controller array
ControllerPtr myControllers[BP32_MAX_GAMEPADS] = { nullptr };

// Initialize the Servo objects
Servo servo1;
Servo servo2;
Servo servo3;
Servo servo4;
Servo servo5;  // Servo 5 object for throttle and brake buttons
```

```
// Flag to track whether a controller is connected
bool controllerConnected = false;


// Function to control motors
void controlMotor(int forwardPin, int backwardPin, int speed) {
  Serial.printf("Controlling motor: forwardPin=%d, backwardPin=%d,
speed=%d\n", forwardPin, backwardPin, speed);


  if (speed > 0) {
    // Move the motor forward
    analogWrite(forwardPin, speed);
    analogWrite(backwardPin, 0);
    Serial.println("Motor moving forward.");
  } else if (speed < 0) {
    // Move the motor backward
    analogWrite(forwardPin, 0);
    analogWrite(backwardPin, -speed);
    Serial.println("Motor moving backward.");
  } else {
    // Stop the motor
    analogWrite(forwardPin, 0);
    analogWrite(backwardPin, 0);
    Serial.println("Motor stopped.");
```

```
    }
}

// Function to stop all motors and servos
void stopAllMotorsAndServos() {
    // Stop all motors
    controlMotor(motor1_forward_pin, motor1_backward_pin, 0);
    controlMotor(motor2_forward_pin, motor2_backward_pin, 0);
    digitalWrite(motor1_enable_pin, LOW);
    digitalWrite(motor2_enable_pin, LOW);
    Serial.println("All motors stopped.");

    // Stop all servos
    servo1.write(90);  // Neutral position
    servo2.write(90);  // Neutral position
    servo3.write(90);  // Neutral position
    servo4.write(90);  // Neutral position
    servo5.write(90);  // Neutral position
    Serial.println("All servos stopped.");
}

// Callback function for connected controller
void onConnectedController(ControllerPtr ctl) {
```

```
  for (int i = 0; i < BP32_MAX_GAMEPADS; i++) {
    if (myControllers[i] == nullptr) {
      Serial.printf("CALLBACK: Controller connected, index=%d\n", i);
      myControllers[i] = ctl;
      controllerConnected = true;
      printBatteryStatus(ctl);
      break;
    }
  }
  digitalWrite(buzzer, HIGH);
  delay(500);
  digitalWrite(buzzer, LOW);
  delay(500);
  digitalWrite(buzzer, HIGH);
  delay(500);
  digitalWrite(buzzer, LOW);
  delay(500);
}

// Callback function for disconnected controller
void onDisconnectedController(ControllerPtr ctl) {
  for (int i = 0; i < BP32_MAX_GAMEPADS; i++) {
    if (myControllers[i] == ctl) {
```

```
      Serial.printf("CALLBACK: Controller disconnected from index=%d\n",
i);
      myControllers[i] = nullptr;
      controllerConnected = false;
      stopAllMotorsAndServos();
      break;
    }
  }
  digitalWrite(buzzer, HIGH);
  delay(500);
  digitalWrite(buzzer, LOW);
  delay(500);
}

// Function to print battery status of a controller
void printBatteryStatus(ControllerPtr ctl) {
  int batteryLevel = ctl->battery();

  if (batteryLevel >= 0) {
    Serial.printf("Controller index=%d battery level: %d%%\n",
ctl->index(), batteryLevel);
  } else {
    Serial.printf("Controller index=%d battery level: Not available\n",
ctl->index());
```

```
  }
}


// Function to move the bot forward by 8 cm and set servo angles simultaneously
void moveBotAndSetServos() {
  // Define the servo target angles
  int targetServo1Angle = 170; // hand left
  int targetServo2Angle = 60; // shoulder
  int targetServo3Angle = 10; // right hand
  int targetServo4Angle = 120;  // right shoulder


  // Define the initial servo angles
  int currentServo1Angle = servo1.read();
  int currentServo2Angle = servo2.read();
  int currentServo3Angle = servo3.read();
  int currentServo4Angle = servo4.read();


  // Define the movement duration and step size
  const int moveDuration = 400; // Adjust this duration to move 8 cm (depends on your bot's speed)
  const int stepDuration = 20;  // Time for each step in milliseconds
  const int steps = moveDuration / stepDuration;  // Number of steps for the movement
```

```
  // Calculate the angle increment for each step
  float servo1Increment = (targetServo1Angle - currentServo1Angle) /
(float)steps;
  float servo2Increment = (targetServo2Angle - currentServo2Angle) /
(float)steps;
  float servo3Increment = (targetServo3Angle - currentServo3Angle) /
(float)steps;
  float servo4Increment = (targetServo4Angle - currentServo4Angle) /
(float)steps;

  // Enable the motors and set speed
  const int motorSpeed = 255; // Set speed
  digitalWrite(motor1_enable_pin, HIGH);
  digitalWrite(motor2_enable_pin, HIGH);
  controlMotor(motor1_forward_pin, motor1_backward_pin, motorSpeed);
  controlMotor(motor2_forward_pin, motor2_backward_pin, motorSpeed);

  // Move the servos and motors simultaneously
  for (int i = 0; i < steps; i++) {
    // Increment servo angles
    currentServo1Angle += servo1Increment;
    currentServo2Angle += servo2Increment;
    currentServo3Angle += servo3Increment;
    currentServo4Angle += servo4Increment;
```

```
    // Write new servo angles
    servo1.write(currentServo1Angle);
    servo2.write(currentServo2Angle);
    servo3.write(currentServo3Angle);
    servo4.write(currentServo4Angle);

    // Delay for the step duration
    delay(stepDuration);
  }

  // Stop the motors after moving forward
  controlMotor(motor1_forward_pin, motor1_backward_pin, 0);
  controlMotor(motor2_forward_pin, motor2_backward_pin, 0);
  digitalWrite(motor1_enable_pin, LOW);
  digitalWrite(motor2_enable_pin, LOW);
}

// Function to perform the BUTTON(triangle) action
void performButtonXAction() {
  // Move forward by 8 cm (assume 2 cm per unit speed)
  digitalWrite(motor1_enable_pin, HIGH);
  digitalWrite(motor2_enable_pin, HIGH);
```

```
  controlMotor(motor1_forward_pin, motor1_backward_pin, motor_range_max / 8);
  controlMotor(motor2_forward_pin, motor2_backward_pin, motor_range_max / 8);
  delay(400); // Adjust the delay to achieve the correct 8 cm movement
  controlMotor(motor1_forward_pin, motor1_backward_pin, 0);
  controlMotor(motor2_forward_pin, motor2_backward_pin, 0);
  digitalWrite(motor1_enable_pin, LOW);
  digitalWrite(motor2_enable_pin, LOW);

  // Set servo angles
  servo1.write(170);  // left hand at 45 degrees
  servo2.write(40);   // right shoulder at 45 degrees
  servo3.write(10);   // left hand at 135 degrees
  servo4.write(140);  // right shoulder at 135 degrees
  delay(1500);
}

// Function to perform the BUTTON_Y action
void performButtonYAction() {
  // Move bot forward by 8 cm and set servo angles simultaneously
  moveBotAndSetServos();
}
```

```
// Function to perform the BUTTON_A(circle) action
void performButtonAAction() {
  // Set servo angles for a Mortal Kombat position
  servo1.write(90);  // Right hand at 60 degrees
  servo2.write(90);  // Shoulder at 90 degrees
  servo3.write(90);  // Left hand at 90 degrees
  servo4.write(90);  // Left shoulder at 90 degrees
}

// Function to perform the BUTTON_B(square) action
void performButtonBAction() {
  // Move forward by 8 cm and set servo angles simultaneously
  moveBotAndSetServos();
}

// Function to process gamepad inputs
void processGamepad(ControllerPtr ctl) {
  if (!ctl->isConnected()) return;

  if (ctl->isPressed(BUTTON_X)) {
    performButtonXAction();
  }
  if (ctl->isPressed(BUTTON_Y)) {
```

```
    performButtonYAction();
  }
  if (ctl->isPressed(BUTTON_A)) {
    performButtonAAction();
  }
  if (ctl->isPressed(BUTTON_B)) {
    performButtonBAction();
  }

  // Read joystick values for servo control
  int leftJoystickY = ctl->getJoystickY(JOYSTICK_LEFT);
  int leftJoystickX = ctl->getJoystickX(JOYSTICK_LEFT);
  int rightJoystickY = ctl->getJoystickY(JOYSTICK_RIGHT);
  int rightJoystickX = ctl->getJoystickX(JOYSTICK_RIGHT);

  // Map joystick values to servo angles (assuming the range is -100 to
100)
  int servo1Angle = map(leftJoystickY, -100, 100, 0, 180);
  int servo2Angle = map(leftJoystickX, -100, 100, 0, 180);
  int servo3Angle = map(rightJoystickY, -100, 100, 0, 180);
  int servo4Angle = map(rightJoystickX, -100, 100, 0, 180);

  // Set servo angles based on joystick input
  servo1.write(servo1Angle);
```

```
  servo2.write(servo2Angle);
  servo3.write(servo3Angle);
  servo4.write(servo4Angle);
}

// Function to process connected controllers
void processControllers() {
  for (int i = 0; i < BP32_MAX_GAMEPADS; i++) {
    if (myControllers[i]) {
      processGamepad(myControllers[i]);
    }
  }
}

// Setup function
void setup() {
  Serial.begin(115200);

  // Set up motor control pins
  pinMode(motor1_enable_pin, OUTPUT);
  pinMode(motor2_enable_pin, OUTPUT);
  pinMode(motor1_forward_pin, OUTPUT);
  pinMode(motor1_backward_pin, OUTPUT);
```

```
  pinMode(motor2_forward_pin, OUTPUT);
  pinMode(motor2_backward_pin, OUTPUT);
  pinMode(buzzer, OUTPUT);

  // Attach servos to their respective pins
  servo1.attach(servo1_pin);
  servo2.attach(servo2_pin);
  servo3.attach(servo3_pin);
  servo4.attach(servo4_pin);
  servo5.attach(servo5_pin); // Attach throttle and brake servo

  // Initialize Bluepad32
  Bluepad32.begin();
  Bluepad32.setOnConnected(onConnectedController);
  Bluepad32.setOnDisconnected(onDisconnectedController);
}

// Main loop
void loop() {
  Bluepad32.update();
  processControllers();
}
```

Output :

The code controls motors and servos using a PS4 controller. Motors are moved forward or backward based on joystick input, and five servos are controlled by the left and right joysticks. Specific actions are mapped to the controller's buttons: the **X** and **Y** buttons move the robot forward by 8 cm and adjust the servos, the **A** button resets all servos to a neutral position, and the **B** button performs the same action as the Y button. The controller's connection status is monitored, and its battery level is displayed. The robot moves forward by 8 cm while simultaneously adjusting the servos, ensuring synchronized movement. Functions are defined to control motors, stop all motors and servos, move the robot and set servos, and perform actions based on button presses. The setup configures the pins for motors and servos and initializes the PS4 controller. In the main loop, inputs are continuously updated to control motors and servos based on the controller's actions.

Step 30 : Explanation

The code controls a robot using a PS4 controller. It moves the robot forward or backward with the joystick and adjusts servos for actions like hand or shoulder movement. Button presses (X, Y, A, B) trigger different actions, like moving forward or resetting the servos. When the controller connects or disconnects, the buzzer provides feedback, and the battery status is shown. The code continuously checks the controller inputs to control the motors and servos accordingly.

Step 31: Boxing Bot code with special moves

Code :

```
#include <Bluepad32.h>

#include <ESP32Servo.h>  // Use ESP32 Servo library


// Define the enable pins for the motors

const int motor1_enable_pin = 13;  // Motor 1 enable pin

const int motor2_enable_pin = 25;  // Motor 2 enable pin

// Define the motor control pins

const int motor1_forward_pin = 12;   // Motor 1 forward pin

const int motor1_backward_pin = 14;  // Motor 1 backward pin
```

```
const int motor2_forward_pin = 27;    // Motor 2 forward pin
const int motor2_backward_pin = 26;   // Motor 2 backward pin
const int buzzer = 15;                // Buzzer pin

// Define the motor control range
const int motor_range_max = 255;    // Maximum speed (forward)
const int motor_range_min = -255;   // Minimum speed (backward)

// Define the servo pins
const int servo1_pin = 2;    // Servo 1 pin (left joystick up and down)
const int servo2_pin = 4;    // Servo 2 pin (left joystick left and right)
const int servo3_pin = 5;    // Servo 3 pin (right joystick up and down)
const int servo4_pin = 18;   // Servo 4 pin (right joystick left and right)
const int servo5_pin = 19;   // Servo 5 pin (throttle and brake buttons)

// Initialize the controller array
ControllerPtr myControllers[BP32_MAX_GAMEPADS] = { nullptr };

// Initialize the Servo objects
Servo servo1;
Servo servo2;
Servo servo3;
Servo servo4;
```

```
Servo servo5;  // Servo 5 object for throttle and brake buttons


// Flag to track whether a controller is connected
bool controllerConnected = false;


// Function to control motors
void controlMotor(int forwardPin, int backwardPin, int speed) {
  Serial.printf("Controlling motor: forwardPin=%d, backwardPin=%d,
speed=%d\n", forwardPin, backwardPin, speed);


  if (speed > 0) {
    // Move the motor forward
    analogWrite(forwardPin, speed);
    analogWrite(backwardPin, 0);
    Serial.println("Motor moving forward.");
  } else if (speed < 0) {
    // Move the motor backward
    analogWrite(forwardPin, 0);
    analogWrite(backwardPin, -speed);
    Serial.println("Motor moving backward.");
  } else {
    // Stop the motor
    analogWrite(forwardPin, 0);
    analogWrite(backwardPin, 0);
```

```
    Serial.println("Motor stopped.");
  }
}

// Function to stop all motors and servos
void stopAllMotorsAndServos() {
  // Stop all motors
  controlMotor(motor1_forward_pin, motor1_backward_pin, 0);
  controlMotor(motor2_forward_pin, motor2_backward_pin, 0);
  digitalWrite(motor1_enable_pin, LOW);
  digitalWrite(motor2_enable_pin, LOW);
  Serial.println("All motors stopped.");

  // Stop all servos
  servo1.write(90);  // Neutral position
  servo2.write(90);  // Neutral position
  servo3.write(90);  // Neutral position
  servo4.write(90);  // Neutral position
  servo5.write(90);  // Neutral position
  Serial.println("All servos stopped.");
}

// Callback function for connected controller
```

```
void onConnectedController(ControllerPtr ctl) {
  for (int i = 0; i < BP32_MAX_GAMEPADS; i++) {
    if (myControllers[i] == nullptr) {
      Serial.printf("CALLBACK: Controller connected, index=%d\n", i);
      myControllers[i] = ctl;
      controllerConnected = true;
      printBatteryStatus(ctl);
      break;
    }
  }
  digitalWrite(buzzer, HIGH);
  delay(500);
  digitalWrite(buzzer, LOW);
  delay(500);
  digitalWrite(buzzer, HIGH);
  delay(500);
  digitalWrite(buzzer, LOW);
  delay(500);
}

// Callback function for disconnected controller
void onDisconnectedController(ControllerPtr ctl) {
  for (int i = 0; i < BP32_MAX_GAMEPADS; i++) {
```

```
    if (myControllers[i] == ctl) {
      Serial.printf("CALLBACK: Controller disconnected from index=%d\n",
i);
      myControllers[i] = nullptr;
      controllerConnected = false;
      stopAllMotorsAndServos();
      break;
    }
  }
  digitalWrite(buzzer, HIGH);
  delay(500);
  digitalWrite(buzzer, LOW);
  delay(500);
}

// Function to print battery status of a controller
void printBatteryStatus(ControllerPtr ctl) {
  int batteryLevel = ctl->battery();

  if (batteryLevel >= 0) {
    Serial.printf("Controller index=%d battery level: %d%%\n",
ctl->index(), batteryLevel);
  } else {
```

```
    Serial.printf("Controller index=%d battery level: Not available\n",
ctl->index());
  }
}


// Function to perform the BUTTON_X action


// Function to move the bot forward by 8 cm and set servo angles
simultaneously
void moveBotAndSetServos() {
  // Define the servo target angles
  int targetServo1Angle = 170; // hand left
  int targetServo2Angle = 60; //shoulder
  int targetServo3Angle = 10; //right hand
  int targetServo4Angle = 120;  //right shoulder               // -edited

  // Define the initial servo angles
  int currentServo1Angle = servo1.read();
  int currentServo2Angle = servo2.read();
  int currentServo3Angle = servo3.read();
  int currentServo4Angle = servo4.read();

  // Define the movement duration and step size
```

```
  const int moveDuration = 400; // Adjust this duration to move 8 cm (depends on your bot's speed)
  const int stepDuration = 20;  // Time for each step in milliseconds
  const int steps = moveDuration / stepDuration;  // Number of steps for the movement

  // Calculate the angle increment for each step
  float servo1Increment = (targetServo1Angle - currentServo1Angle) / (float)steps;
  float servo2Increment = (targetServo2Angle - currentServo2Angle) / (float)steps;
  float servo3Increment = (targetServo3Angle - currentServo3Angle) / (float)steps;
  float servo4Increment = (targetServo4Angle - currentServo4Angle) / (float)steps;

  // Enable the motors and set speed
  const int motorSpeed = 255; // Set speed
  digitalWrite(motor1_enable_pin, HIGH);
  digitalWrite(motor2_enable_pin, HIGH);
  controlMotor(motor1_forward_pin, motor1_backward_pin, motorSpeed);
  controlMotor(motor2_forward_pin, motor2_backward_pin, motorSpeed);

  // Move the servos and motors simultaneously
  for (int i = 0; i < steps; i++) {
    // Increment servo angles
```

```
    currentServo1Angle += servo1Increment;
    currentServo2Angle += servo2Increment;
    currentServo3Angle += servo3Increment;
    currentServo4Angle += servo4Increment;

    // Write new servo angles
    servo1.write(currentServo1Angle);
    servo2.write(currentServo2Angle);
    servo3.write(currentServo3Angle);
    servo4.write(currentServo4Angle);

    // Delay for the step duration
    delay(stepDuration);
  }

  // Stop the motors after moving forward
  controlMotor(motor1_forward_pin, motor1_backward_pin, 0);
  controlMotor(motor2_forward_pin, motor2_backward_pin, 0);
  digitalWrite(motor1_enable_pin, LOW);
  digitalWrite(motor2_enable_pin, LOW);
}

// Function to perform the BUTTON(triangle) action
```

```
void performButtonXAction() {
  // Move forward by 8 cm (assume 2 cm per unit speed)
  digitalWrite(motor1_enable_pin, HIGH);
  digitalWrite(motor2_enable_pin, HIGH);
  controlMotor(motor1_forward_pin, motor1_backward_pin, motor_range_max / 8);
  controlMotor(motor2_forward_pin, motor2_backward_pin, motor_range_max / 8);
  delay(400); // Adjust the delay to achieve the correct 8 cm movement
  controlMotor(motor1_forward_pin, motor1_backward_pin, 0);
  controlMotor(motor2_forward_pin, motor2_backward_pin, 0);
  digitalWrite(motor1_enable_pin, LOW);
  digitalWrite(motor2_enable_pin, LOW);

  // Set servo angles
  servo1.write(170);  // left hand at 45 degrees
  servo2.write(40);   // right shoulder at 45 degrees
  servo3.write(10);   // left hand at 135 degrees
  servo4.write(140);  // right shoulder at 135 degrees -edited
  delay(1500);
}

// Function to perform the BUTTON_Y action
void performButtonYAction() {
```

```
  // Move bot forward by 8 cm and set servo angles simultaneously
  moveBotAndSetServos();
}

// Function to perform the BUTTON_A(circle) action
void performButtonAAction() {
  // Set servo angles for a Mortal Kombat position
  servo1.write(90);  // Right hand at 60 degrees
  servo2.write(90); // Left hand at 120 degrees
  servo3.write(90); // Right shoulder at 180 degrees
  servo4.write(90);  // Left shoulder at 90 degrees
  servo5.write(45);  // Move hip 45 degrees left
  servo2.write(40); // Right hand at 60 degrees
  servo1.write(180); // Left hand at 120 degrees
  delay(400);
  servo1.write(90);  // Right hand at 60 degrees
  servo2.write(90); // Left hand at 120 degrees
  servo3.write(90); // Right shoulder at 180 degrees
  servo4.write(90);  // Left shoulder at 90 degrees
  servo5.write(90);  // Move hip 45 degrees left
  delay(100);
  servo5.write(135);
  servo3.write(0); // Right shoulder at 180 degrees
```

```
  servo4.write(130);
  delay(400);
  servo1.write(90);  // Right hand at 60 degrees
  servo2.write(90); // Left hand at 120 degrees
  servo3.write(90); // Right shoulder at 180 degrees
  servo4.write(90);  // Left shoulder at 90 degrees
  servo5.write(90);
  delay(100);
  servo5.write(45);
  servo2.write(40); // Right hand at 60 degrees
  servo1.write(180); // Left hand at 120 degrees
  delay(400);
  servo1.write(90);  // Right hand at 60 degrees
  servo2.write(90); // Left hand at 120 degrees
  servo3.write(90); // Right shoulder at 180 degrees
  servo4.write(90);  // Left shoulder at 90 degrees
  servo5.write(90);  // Move hip 45 degrees left
  delay(100);
  servo5.write(135);
  servo3.write(0); // Right shoulder at 180 degrees
  servo4.write(130);
  delay(400);
}
```

```
// Function to perform the BUTTON_B(square) action
void performButtonBAction() {
  // john cena
  servo1.write(90);  // Right hand at 60 degrees
  servo2.write(90); // Left hand at 120 degrees
  servo3.write(90); // Right shoulder at 180 degrees
  servo4.write(90);  // Left shoulder at 90 degrees
  servo5.write(45);  // Move hip 45 degrees left
  servo2.write(40);
  delay(800);  // Right hand at 60 degrees
  servo1.write(180); // Left hand at 120 degrees
  delay(400);
  servo1.write(140); // Right hand at 60 degrees
  delay(400);
  servo1.write(180); // Left hand at 120 degrees
  delay(400);
  servo1.write(140); // Right hand at 60 degrees
  delay(400);
}

// Process gamepad input to control motors and servos
void processGamepad(ControllerPtr ctl) {
```

```
// Check if BUTTON_X is pressed
if (ctl->a()) {
  Serial.println("BUTTON_X pressed: Performing action.");
  performButtonXAction();
  return;
}

// Check if BUTTON_Y is pressed
if (ctl->y()) {
  Serial.println("BUTTON_Y pressed: Performing action.");
  performButtonYAction();
  return;
}

// Check if BUTTON_A is pressed
if (ctl->b()) {
  Serial.println("BUTTON_A pressed: Performing action.");
  performButtonAAction();
  return;
}

// Check if BUTTON_B is pressed
if (ctl->x()) {
```

```
    Serial.println("BUTTON_B pressed: Performing action.");
    performButtonBAction();
    return;
  }

  // Check the state of the D-pad buttons
  uint8_t dpadState = ctl->dpad();

  // Handle D-pad input for movement control
  if (dpadState & DPAD_UP) {
    digitalWrite(motor1_enable_pin, HIGH);
    digitalWrite(motor2_enable_pin, HIGH);
    Serial.println("DPAD UP pressed: Moving motors forward.");
    controlMotor(motor1_forward_pin, motor1_backward_pin,
motor_range_max);
    controlMotor(motor2_forward_pin, motor2_backward_pin,
motor_range_max);
  } else if (dpadState & DPAD_DOWN) {
    digitalWrite(motor1_enable_pin, HIGH);
    digitalWrite(motor2_enable_pin, HIGH);
    Serial.println("DPAD DOWN pressed: Moving motors backward.");
    controlMotor(motor1_forward_pin, motor1_backward_pin,
motor_range_min);
    controlMotor(motor2_forward_pin, motor2_backward_pin,
motor_range_min);
```

```
  } else if (dpadState & DPAD_LEFT) {
    digitalWrite(motor1_enable_pin, HIGH);
    digitalWrite(motor2_enable_pin, HIGH);
    Serial.println("DPAD LEFT pressed: Turning left.");
    controlMotor(motor1_forward_pin, motor1_backward_pin,
motor_range_min);
    controlMotor(motor2_forward_pin, motor2_backward_pin,
motor_range_max);
  } else if (dpadState & DPAD_RIGHT) {
    digitalWrite(motor1_enable_pin, HIGH);
    digitalWrite(motor2_enable_pin, HIGH);
    Serial.println("DPAD RIGHT pressed: Turning right.");
    controlMotor(motor1_forward_pin, motor1_backward_pin,
motor_range_max);
    controlMotor(motor2_forward_pin, motor2_backward_pin,
motor_range_min);
  } else {
    digitalWrite(motor1_enable_pin, LOW);
    digitalWrite(motor2_enable_pin, LOW);
    Serial.println("No DPAD button pressed: Stopping both motors.");
    controlMotor(motor1_forward_pin, motor1_backward_pin, 0);
    controlMotor(motor2_forward_pin, motor2_backward_pin, 0);
  }


  // Read the left joystick axes
```

```
int left_axis_x = ctl->axisX();
int left_axis_y = ctl->axisY();

// Read the right joystick axes
int right_axis_x = ctl->axisRX();
int right_axis_y = ctl->axisRY();

// Map the joystick values to servo angle ranges
int servo1_angle = map(left_axis_y, 511, -512, 0, 180);
int servo2_angle = map(left_axis_x, 511, -512, 0, 180);
int servo3_angle = map(right_axis_y, -511, 512, 0, 180);
int servo4_angle = map(right_axis_x, 511, -512, 0, 180);

// Control servos based on joystick inputs
servo1.write(servo1_angle);
servo2.write(servo2_angle);
servo3.write(servo3_angle);
servo4.write(servo4_angle);

// Control Servo 5 using throttle and brake buttons
if (ctl->brake()) {
  servo5.write(180);
  Serial.println("Brake button pressed: Servo5 set to 0 degrees.");
```

```
  } else if (ctl->throttle()) {
    servo5.write(0);
    Serial.println("Throttle button pressed: Servo5 set to 180 degrees.");
  } else {
    servo5.write(90);
  }

  // Print battery status
  printBatteryStatus(ctl);
}

// Process all controllers
void processControllers() {
  for (auto myController : myControllers) {
    if (myController && myController->isConnected() &&
myController->hasData()) {
      if (myController->isGamepad()) {
        processGamepad(myController);
      } else {
        Serial.println("Controller type not supported");
      }
    }
  }
}
```

```
// Setup function
void setup() {
  Serial.begin(115200);
  Serial.printf("Firmware: %s\n", BP32.firmwareVersion());

  // Set up the motor control pins as outputs
  pinMode(motor1_forward_pin, OUTPUT);
  pinMode(motor1_backward_pin, OUTPUT);
  pinMode(motor2_forward_pin, OUTPUT);
  pinMode(motor2_backward_pin, OUTPUT);
  pinMode(motor1_enable_pin, OUTPUT);
  pinMode(motor2_enable_pin, OUTPUT);
  pinMode(buzzer, OUTPUT);


  // Ensure motors are stopped initially
  digitalWrite(motor1_enable_pin, LOW);   // Add this line
  digitalWrite(motor2_enable_pin, LOW);   // Add this line
  analogWrite(motor1_forward_pin, 0);     // Add this line
  analogWrite(motor1_backward_pin, 0);    // Add this line
  analogWrite(motor2_forward_pin, 0);     // Add this line
  analogWrite(motor2_backward_pin, 0);    // Add this line
```

```
  // Attach the servo objects to their respective pins
  servo1.attach(servo1_pin);
  servo2.attach(servo2_pin);
  servo3.attach(servo3_pin);
  servo4.attach(servo4_pin);
  servo5.attach(servo5_pin);

  // Initialize Bluepad32 and set callback functions
  BP32.setup(&onConnectedController, &onDisconnectedController);

  Serial.println("Setup completed. Waiting for controllers...");
}

// Main loop function
void loop() {
  // Update the gamepad data
  BP32.update();

  // Process connected controllers
  processControllers();
}
```